

Yenepoya (Deemed to be University).
Project – Low Level Design
on
AI-Based Phishing Detection for Retail

Student Name:

Aishah Syed

Industry Mentor: Mr. Shashank Korram

Guided by: Mr. Vikyath

Table of Contents

1. Introduction	3
1.1 Scope of the document	3
1.2 Intended Audience	3
1.3 System overview	3
2. System Design	4
2.1. Application Design	4
2.2. Process Flow	4
2.3. Information Flow	5
2.4. Components Design	6
2.5. Key Design Considerations	7
2.6. API Catalogue	7
3. Data Design	8
3.1. Data Model	8
3.2. Data Access Mechanism	9
3.3. Data Retention Policies	9
4. Interfaces	9
5. Non-Functional Requirements	10
5.1. Security Aspects.....	10
5.2. Performance Aspects	10
6. References	11

Low-Level Design (LLD): AI-Based Phishing Detection for Retail

1. INTRODUCTION

1.1. SCOPE OF THE DOCUMENT

This Low-Level Design (LLD) document serves as the primary technical reference and foundational blueprint for the AI-Based Phishing Detection for Retail system. It details the internal logic, data structures, and algorithmic configurations required for system implementation, specifically covering data cleaning, Natural Language Processing (NLP) feature extraction, and the Multinomial Naive Bayes classifier. This document defines the technical specifications necessary for the development, testing, and deployment of the classification engine, ensuring a robust workflow for identifying retail-themed phishing threats.

1.2. INTENDED AUDIENCE

Developers, Data Scientists, and Security Analysts responsible for maintaining or extending the phishing detection model.

1.3. SYSTEM OVERVIEW

The AI-Based Phishing Detection system is a supervised machine learning pipeline engineered to automate the identification of retail-themed phishing threats. By analyzing linguistic patterns, the system distinguishes between legitimate communications (Ham) and deceptive messages (Spam) such as fraudulent invoices or fake reward notifications.

The operational lifecycle consists of four streamlined phases:

1. **Data Preprocessing & Cleaning:** The system identifies missing information and duplicate records, this ensures that the AI learns from the unique examples, rather than memorizing the repeated spam messages.
2. **Feature Extraction (Bag-of-Words):** Using the CountVectorizer module, the raw text is transformed into a numerical form. The process includes tokenization and English stop-word removal, limiting the vocabulary to the top 1,000 most significant retail-relevant features for optimal performance.
3. **Probabilistic Classification:** The core engine utilizes a Multinomial Naive Bayes classifier. By applying Bayes' Theorem, the algorithm calculates the mathematical probability of an email being malicious based on learned word frequencies, making it highly efficient for high-dimensional text data.
4. **Inference and Confidence Scoring:** Finally, the system assigns a label to the email and provides a Confidence Score (e.g., 99%). This score indicates how certain the AI is about its decision. High-confidence scams can be blocked automatically, while low-confidence messages can be flagged for human review, reducing the chance of blocking a legitimate email by mistake.

2.SYSTEM DESIGN

2.1. APPLICATION DESIGN

The application is designed as a modular pipeline in Python. It utilizes Scikit-Learn for the core ML logic and Pandas for data manipulation.

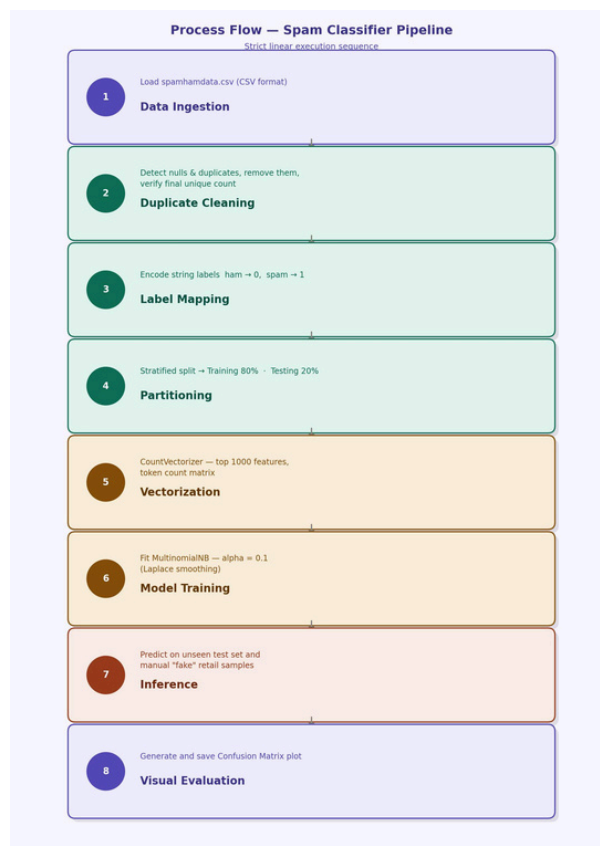
Core Usage of Pandas & Skit-Learn:

- Pandas: Used for high-performance data manipulation. It handles the "Dataframe" structures, allowing the system to clean, filter, and map labels (such as converting 'spam' to 1) with minimal code complexity.
- Scikit-Learn: Provides the mathematical engine for the project. The system uses this library for splitting the data, extracting text features, and running the Multinomial Naive Bayes algorithm.

2.2. PROCESS FLOW

The execution follows a strict linear sequence:

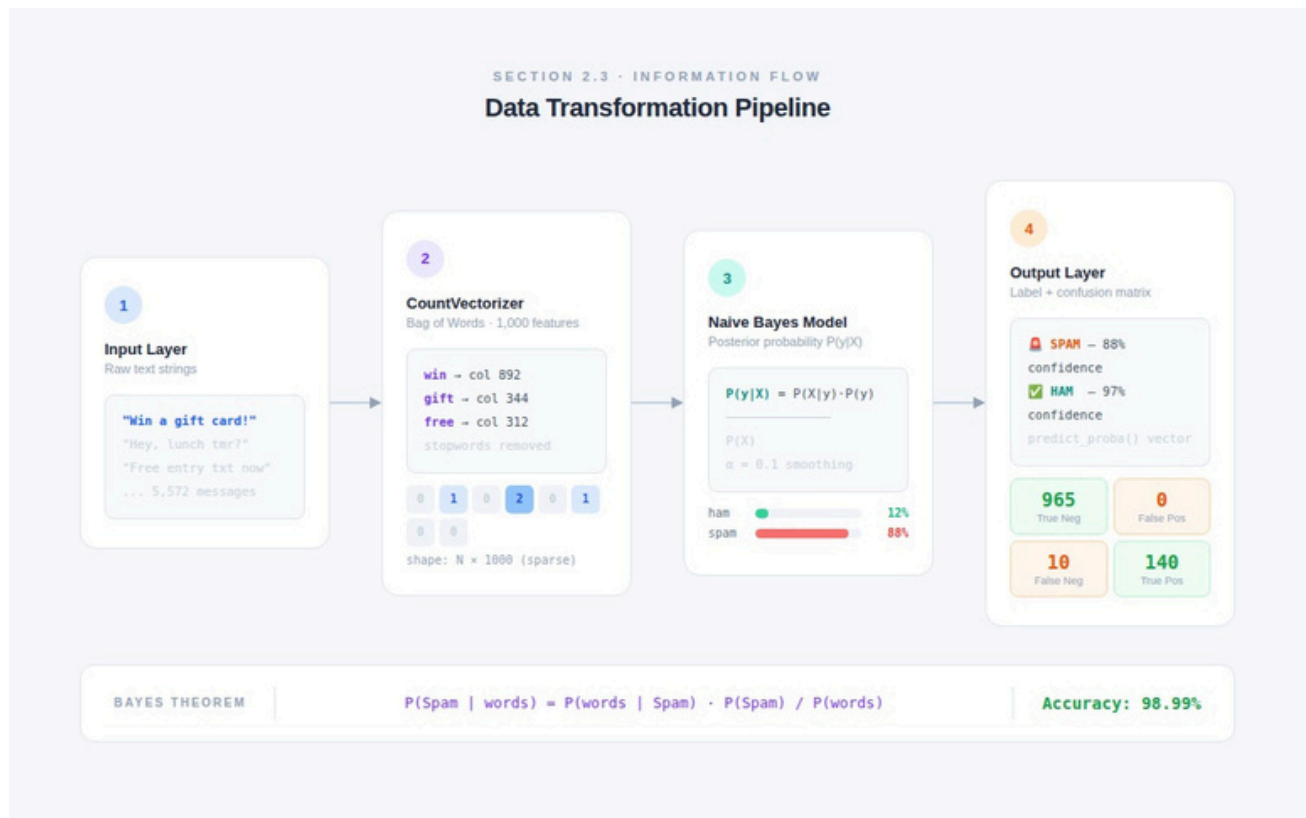
1. Data Ingestion: Load spamhamdata.csv (CSV format).
2. Duplicate Cleaning: Detect initial null values and duplicates, remove them, and verify the final unique count.
3. Label Mapping: Encode string labels (ham/spam) to integers (0 and 1).
4. Partitioning: Split data into Training (80%) and Testing (20%) using stratification.
5. Vectorization: Apply CountVectorizer to generate token counts for the top 1000 features.
6. Model Training: Fit the MultinomialNB model ($\alpha = 0.1$).
7. Inference: Run predictions on unseen test data and manual "fake" retail samples.
8. Visual Evaluation: Generate and save a Confusion Matrix plot.



2.3. INFORMATION FLOW

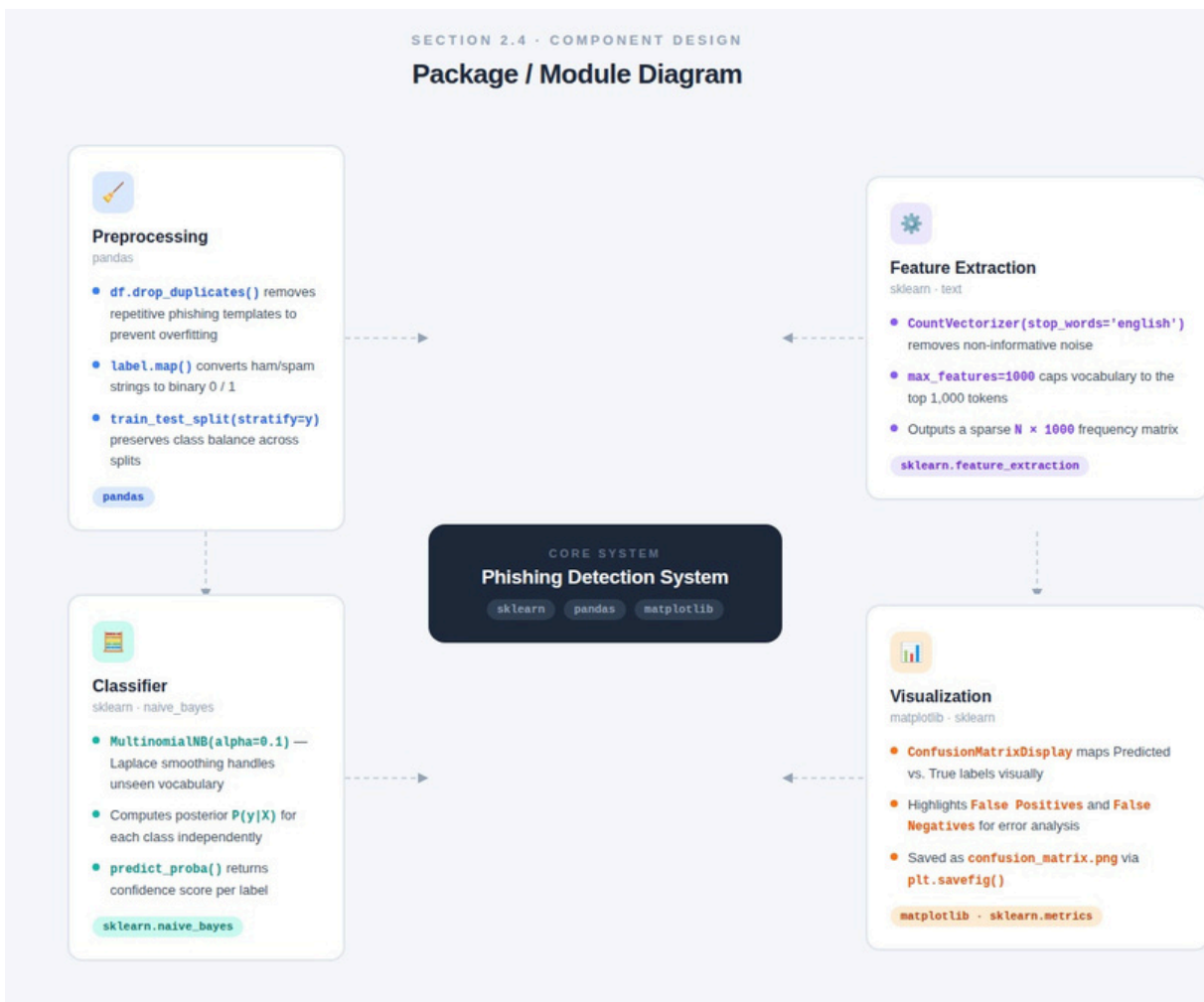
The system processes data through a linear pipeline, transforming unstructured natural language into a structured probabilistic classification.

- **Input Layer:** The process begins with N rows of raw text strings. This data is non-uniform and contains linguistic noise like punctuation, casing, and common articles.
- **Transformation (Feature Engineering):** The CountVectorizer performs a "Bag of Words" transformation. It maps the text into a high-dimensional Sparse Matrix where each row represents a message and each column represents the frequency of a specific word from the 1,000-word vocabulary.
- **Stochastic Computation:** The model calculates the posterior probability $P(y|X)$ —the likelihood that a label y is true given the feature set X .
- **Output Layer:** The final output is a probability vector for classification and a Graphical Confusion Matrix that provides a visual representation of accuracy, highlighting the model errors.



2.4. COMPONENT DESIGN

- Preprocessing Module: Uses `df.drop_duplicates(subset=['text'])` to ensure the model does not overfit to repetitive phishing templates.
- Feature Extraction Module: Uses `CountVectorizer` with `stop_words='english'` to remove non-informative noise.
- Classification Module: Uses `MultinomialNB`. The Laplace smoothing parameter ($\alpha = 0.1$) ensures the model handles new words gracefully.
- Visualization Module: Uses `ConfusionMatrixDisplay` to map Predicted vs. True labels, highlighting False Positives and False Negatives.



2.5. KEY DESIGN CONSIDERATIONS

These specific design choices were implemented to ensure the model is robust and statistically strong:

- **Stratification (Class Balance Preservation):** By utilizing `stratify=y` during the `datasplit`, the system ensures that the training and the testing data has same ratio of "spam" and "ham". This is critical because spam often takes up a lesser portion. Without stratification, the model will fail to learn the patterns effectively.
- **Probability Thresholds & Confidence Scoring:** Rather than returning simple "spam/ham" label, the system utilizes `predict_proba()` which tells the probability across classes. By quantifying the the model's confidence e.g., 99.8% vs 51%, the system can prioritize high-confidence threats or flag low-confidence message for manual review.
- **Laplace Smoothing:** The model uses a smoothing parameter ($\alpha=0.1$) to handle zero-probability cases. This ensures if a new phishing email contains a word which isn't in the vocabulary, the mathematical probability shouldn't fall to zero, allowing for better generalisation on unseen data.

2.6. API CATALOGUE

This section lists the specific library functions utilized to build the system and their respective roles in the information process.

Function	Library	Purpose
<code>pd.read_csv()</code>	Pandas	Data Ingestion: Loads the TSV dataset into a structured DataFrame.
<code>df.drop_duplicates()</code>	Pandas	Data Integrity: Identifies and removes redundant text samples to prevent overfitting.
<code>train_test_split()</code>	Scikit-Learn	Partitioning: Separates data into 80% Training and 20% Testing sets with stratification.
<code>CountVectorizer.fit_transform()</code>	Scikit-Learn	Vectorization: Learns the vocabulary and converts text into a numerical sparse matrix.
<code>MultinomialNB.fit()</code>	Scikit-Learn	Model Training: Executes the Naive Bayes algorithm on the training sparse matrix.
<code>model.predict_proba()</code>	Scikit-Learn	Inference: Calculates the posterior probability for each test email.
<code>ConfusionMatrixDisplay.from_predictions()</code>	Scikit-Learn	Reporting: Generates the heatmap comparing actual labels to predicted values.
<code>plt.savefig()</code>	Matplotlib	Persistence: Exports the generated graphical results to a .png file for documentation.

3. DATA DESIGN

3.1. DATA MODEL

Column	Data Type	Description
label	String	Original category ('ham' or 'spam')
text	String	Raw email/SMS content
label_num	Integer	Encoded label (0 for Ham, 1 for Spam)

3.2. DATA ACCESS MECHANISM

This module serves as the Data Access Object (DAO) layer, responsible for establishing a stream between the persistent storage and the application memory.

- **Data Source:** The system utilizes a local flat-file database stored in .csv format (data/spamhamdata.csv).
- **Parsing Protocol:** Since the raw data contains commas within the text messages, a standard comma-separated parser would fail. The mechanism is configured to use a Tab-Separated Values (TSV) protocol.
- **Delimiter Configuration:** `sep='\t'` is explicitly defined to treat tabs as column boundaries.
- **Schema Definition:** As the source file lacks a header row, the `names=['label', 'text']` parameter is passed to manually inject a schema during the ingestion phase.
- **Memory Management:** Upon execution of `pd.read_csv()`, the data is loaded into a Pandas DataFrame object. This allows for high-speed, in-memory vectorized operations during the preprocessing stage.

3.3. DATA RETENTION POLICIES

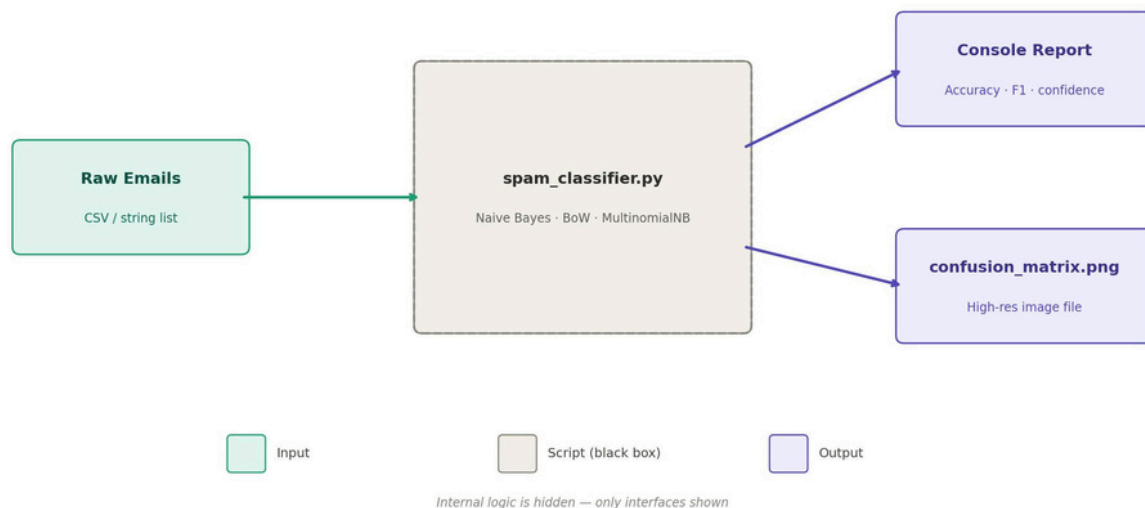
This section defines the lifecycle and persistence of data within the systems, before and after its execution.

- **Volatile Memory (RAM):** The processed sparse matrix (X) along with the derived weights of multinomial Naive Bayes model are held in system memory only during execution session. This allows high speed I/O execution without overhead.
- **Persistent Storage:** The final evaluation which is the “confusion_matrix.png” is a high resolution image which can be used for post-mortem analysis and documentation without having to re-run the training pipeline.
- **Transient State:** Raw input strings are discarded immediately after the vectorization, minimizing the digital footprints and adhering to privacy policies by not storing texts for a longer time than necessary

4.INTERFACES

The system is designed with a clear separation between data input, internal logic, and multi-format reporting.

- **Input Interface** The system accepts raw text strings or batch CSV files as shown in the Raw Emails entry point. This interface allows for both large-scale historical datasets and individual test cases, ensuring the system is versatile for both training and live detection.
- **Textual Output (Console)** As indicated by the Console Report output, the system provides real-time feedback through the command-line. It displays essential performance metrics like Accuracy and F1-Scores, along with "Confidence Levels" for individual emails to quantify the model's certainty.
- **Visual Output Interface** The primary analytical output is the confusion_matrix.png file. This high-resolution image allows users to visualize the model's reliability by mapping correct classifications against errors (False Positives and False Negatives) in a professional heatmap format.



5. NON-FUNCTIONAL REQUIREMENTS

5.1. SECURITY ASPECTS

- **Local-Only Processing:** The architecture is designed for "On-Premise" execution. Because no data is transmitted to external APIs or cloud services, sensitive organizational communication remains within the local secure environment.
- **Obfuscation Resistance:** The Feature Extraction Module serves as a natural security filter. By stripping punctuation and converting text to a normalized vocabulary, the system effectively ignores common "obfuscation" tactics used by attackers, such as using special characters (e.g., "W_I_N_N_E_R") to bypass simple keyword filters.

5.2. PERFORMANCE ASPECTS

- **Computational Efficiency:** The Multinomial Naive Bayes algorithm was selected specifically for its $O(n)$ complexity. It provides near-instantaneous inference even on lower-end hardware, making it ideal for real-time mail filtering.
- **Operational Reliability:** Through the use of Stratification and Duplicate Removal, the system ensures high "Generalization" capabilities. This means the model remains reliable when encountering new, unseen phishing variants rather than simply memorizing the training templates.

6. REFERENCES

- Scikit-Learn (Multinomial Naive Bayes):
https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
- Pandas (Duplicate Handling):
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html
- Matplotlib (Image Persistence/Saving):
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.savefig.html
- Scikit-Learn (Confusion Matrix Display):
<https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ConfusionMatrixDisplay.html>